



8

4

480 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

التوابع 2

د.مدحت الصوص د.هبة حاتم



RB Informatics; 05/12/2024

برمجة C++

في المحاضرة السابقة قلنا أننا نعرف التابع الرئيسي main بعد جميع التوابع التي نعرفها ، في هذه المحاضرة نقول أنه ليس من الضروري تعريف التابع main بعد جميع التوابع ولدينا بعض الأمثلة :

☐ f1 ()

☐ f2 ()

ترويصة header

الجسم body

في المثال السابق لا يمكننا استدعاء f_2 في f_1 (يعطي error في هذه اللغة)، لأنه يقوم بإجراء مسحة واحدة للكود ... أي أنه عندما قرأ f_1 لم يجد f_2 قبله بينما لو كان الأمر بشكل عكسي فلا يوجد مشكلة.

ماذا أفعل إذا كنت أريد استدعاء f_2 في f_1 ؟

إما أن أعرف f_2 قبل f_1 أو أن أصرح عنه قبل f_1 ، أي اكتب مثلاً `void f3(int x)` قبل f_1 ، ثم أعرف عن f_2 بكتابة الترويسة مع الجسم.

ما الفرق بين التصريح والتعريف؟؟

1. التصريح declaration:

```
int fun1(int x);
```

```
void fun2(int x)
```

```
{
    fun1(x);
}
```

2. التعريف definition:

```
int fun1(int x)
```

```
{
    return x;
}
```

ملاحظة لا نحتاج أن نصرح عن التابع قبل استخدامه في أغلب لغات البرمجة ما عدا ++c وهذا لأنها تقوم فقط بفحص واحد للكود عند البناء.
بينما اللغات الأخرى تقوم بفحص مضاعف لتخطي هذه المشكلة.

المتحولات

1. local variables (متحولات محلية): تموت عند انتهاء وظيفتها. مثال:

```
void f1 (int x)
{
    int y = 0;
    if (...)
    {
        int z;
    }
    for (...)
    {
        int z;
    }
}
```

هنا ولد المتحول z ثم مات

هنا ولد المتحول z ثم مات



2. global variables (متحولات عامة): لا تموت طوال عمل البرنامج، وتعرف خارج جميع التوابع (حتى التابع main).

جميع توابع البرنامج ترى هذا النوع من المتحولات على أنه علبة واحدة، ولها إمكانية للوصول إليها وقراءة محتوى العلبة والتعديل عليها.

التخزين

هناك نوعين من الذاكرة التي تخزن فيها المتحولات:

stack	heap
كلاهما من الذاكرة RAM	
ثابتة لكامل البرنامج وتكون بالميجابايت مثلاً 32MB	الذاكرة المتبقية من RAM
لمتحويلات المحلية تعرف ضمنها	المتحويلات العامة تعرف ضمنها

بسبب حجم الذاكرة stack يجب الانتباه إلى المتحولات التي نعرفها، فمثلاً إذا أردت تعريف مصفوفة فيها عشرة ملايين خانة من النوع int، فإنها تأخذ مساحة قدرها 40 مليون بايت تقريباً، أي 40 ميغا بايت، وبالتالي تكون الذاكرة stack غير كافية (في هذه الحالة إذا كانت stack 32 ميغابايت) وسيظهر خطأ stack over flow، فنعرف المصفوفة على أنها global كي نتخزن في الذاكرة heap.

كيفية كتابة توابع لمصفوفات :

```
void print (int arr[])
```

لا داعي لوضع عدد عناصر المصفوفة ولا مانع من وضعها،
لكن يفضل عدم وضعه.

يكون استدعاء التابع السابق (print) في التابع الأساسي (main):

```
int a[5];  
print (a);
```

إذا أردنا تابع يقوم بطباعة جميع عناصر المصفوفة:

```
void print (int arr[], int n)  
{  
    for (int i=0; i<n; i++)  
        cout << arr[i] << endl;  
}
```

ويكون الاستدعاء في التابع (main):

```
int a[5];  
print (a, 5);
```

تابع يقوم بطباعة جميع عناصر المصفوفة (هنا مصفوفة ثنائية):

```
void Print (a[][4], int n)  
{  
    for (int j=0; j<4; j++)  
        for (int i=0; i<n; i++)  
            cout << a[i][j] << endl;  
}
```

ملاحظة هامة: عدد الأعمدة في المصفوفة الثانية يجب أن يحدد عند كتابة التابع، بينما لا يهم وجود عدد الأسطر.

يجب على المستخدم عند استدعاء التابع أن يلتزم بعدد الأعمدة نفسه في المثال السابق (4).

- إذا قمنا بوضع كلمة static قبل تعريف أي متحول، فإن قيمته ستبقى موجودة في الذاكرة ولن تُحذف (أي أنه متحول محلي لكن لن يموت). مثال:

```
void fun( ... )  
{  
    static int n = 5;  
    n++;  
}
```

الأسطر السابقة تعني أنه إذا تحقق الشرط، سيتولد متحول جديد من النوع int وله قيمة 5، ثم ستتم زيادة قيمته بواحد لتصبح 6، وتبقى قيمته في الذاكرة يستخدم مثلاً لعدد مرات استخدام التابع لأنه يعرف المتحول أول مرة فقط ثم يقوم بالتعامل مع القيمة المخزنة في الذاكرة.

التحميل الزائد Function overloading

التحميل الزائد هو أن أعرف أكثر من تابع بنفس الاسم.

مثال:

```
int add (int a, int b)
{
    return a+b;
}

int add (int a, int b, int c)
{
    return a+b+c;
}
```

من الممكن تعريف أكثر من تابع بنفس الاسم لكن ضمن شروط:

1. إما أن يختلف عدد المتحولات لكل تابع عن الآخر (الأول لديه a و b، أما الثاني فلديه a و b و c).
2. أو يختلف نمط المتحولات لكل تابع عن الآخر (char, float, int, ...).
3. لا علاقة لتغيير أسماء المتحولات، فمثلاً العدد أو النمط يجب أن يكونا مختلفين.

▪ لكل تابع توقيع خاص به، والتوقيع يكون: اسم التابع وأنماط متحولاته.

فالتابع الأول توقيعهُ add_int_int

والتابع الثاني توقيعهُ: add_int_int_int

خلاصة الفكرة السابقة أنه يجب اختلاف توقيع كل تابع عن توقيع أي تابع آخر.

الفكرة الأخيرة في هذه المحاضرة:

لدينا البرنامج التالي:

```
int add (float a, int b)
{
    return ...
}

int add (int a, float b)
{
    return ...
}
```

عند الاستدعاء لا يمكن كتابة (3, 2) add لأنه استدعاء غامض.

الحاسوب لن يعلم أي تابع قمت باستدعائه (إذا قمنا بالاستدعاء الغامض سيعطينا error).

Good luck

See you in the last
lecture

